

1. (14 points) Use the radix sort algorithm to sort the following numbers. Assume that every data is 3-digit integer.

456, 329, 478, 59, 52, 447, 380, 126, 237, 599, 626, 255.

- a) Draw a figure to show the actions step by step (see example figure in slide 50 or 54 of Ch6-sorting-heap-linear lecture notes) by treating each digit as a “digit”. (5 points)

456		380		126		52
329		52		626		59
478		255		329		126
59		456		237		237
52		126		447		255
447	→	626	→	52	→	329
380		447		255		380
126		237		456		447
237		478		59		456
599		329		478		478
626		59		380		599
255		599		599		626
col0		col2		col3		col4

- b) Explain why stable sorting at each step is important. You just need to state that correctness cannot be guaranteed (by giving an example) if you did not apply stable sorting at that step (5 points).

This could have many answers. e.g. in the most significant digit step (which is used to derive col4), for 329 and 380, both has the most significant digit as 3, if we do not keep the original order of 29 and 80, we could get 380 and 329.

- c) Describe what conditions should be met for radix sort to be $O(n)$? (4 points)

When d (each key has at most d digits, which is 3 in the question) is a constant and $k = O(n)$ (k is the range of each radix), Radix-sort takes $O(n)$ time.

2. (14 points) Suppose we want to apply Radix Sort to sort 100,000 4-letter words with each letter taken from the English alphabet (26 letters, all lower cases). Assuming that the running time for sorting n elements within range $1..k$ using Counting Sort is $2n+2k$, calculate the running time for each of the following strategies a), b) and c). Show the justification.

- a) treat letters at each of the four positions as a digit. (4 points)
b) treat 2-letter subwords at positions 1-2 as a digit and 2-letter subwords at positions 3-4 as another digit. (4 points)
c) treat all 4 letters as a digit. (4 points)

d) which strategy is the best strategy to minimize the running time? (2 points)

Answer: Let's compute the running time for each case. For all cases, $n = 100,000$.

- (1) In this case, $k = 26$ (we have 26 letters in English alphabet) and $d = 4$ (the number of digits is 4). The running time is $d * (2n + 2k) = 4 (2 * 100000 + 2 * 26) = 800208$.
- (2) In this case, $k = 26 * 26 = 676$ and $d = 2$. The running time is $d * (2n + 2k) = 2 (2 * 100000 + 2 * 676) = 402704$.
- (3) In this case, $k = 26^4 = 456976$ and $d = 1$. The running time is $d * (2n + 2k) = 1 * (2 * 100000 + 2 * 456976) = 1113952$.
- (4) Clearly the second case leads to the lowest running time. In other words, when treating 2-letter subwords as a digit is the best strategy to minimize the running time when using Radix-sort for sorting 100000 4-letter words.